

Contents

1 Provisioning	1
1.1 Hosting provider	1
1.2 VPS system/specifications	1
1.3 Provisioning the server	1
1.4 Bootstrapping	1

1 Provisioning

1.1 Hosting provider

We chose to rent a VPS from Hetzner due to its strong balance between affordability and performance. Hetzner is well known for offering cost-effective virtual private servers with reliable hardware specifications, making it a suitable choice for hosting our application within a limited budget.

Another important factor was Hetzner’s built-in firewall functionality. This allowed us to define firewall rules at the infrastructure level, meaning that unwanted traffic could be filtered before it ever reached the server hardware itself. Configuring firewall rules at this level improves security by reducing the server’s direct exposure to unwanted traffic. After finishing the migration and setup of related applications, we updated our Go provisioning script to also create the firewall and attach it to the server.

1.2 VPS system/specifications

Converting the legacy system from Python to Go significantly improved performance, with faster request handling, lower CPU usage, and reduced memory consumption. This allowed the application to run efficiently on less powerful hardware while handling a similar workload. The biggest gains came when the server was dedicated solely to the compiled Go application, but due to budget constraints, all services had to share a single server.

We chose a Hetzner CX23 server with 2 vCPUs, 4 GB RAM, and 40 GB storage. Multiple vCPUs were necessary to support concurrent applications, while 4 GB RAM ensured stable performance without heavy swap usage. The server also includes 20 TB monthly egress, offering strong cost efficiency at just €3.99 per month.

1.3 Provisioning the server

We chose to define our infrastructure provisioning logic as code rather than relying on Hetzner’s GUI. Hetzner provides an official Go library that acts as a wrapper around their API, which we used to automate the provisioning of the server directly from our application code.

Terraform was also considered as a potential solution for infrastructure provisioning. However, we ultimately preferred the simplicity and maintainability of using a single Go script. This decision aligned naturally with the broader migration of the main application from Python to Go, allowing the team to remain within a familiar ecosystem and programming language.

Adopting Terraform or similar tools would have introduced additional complexity, requiring the team to learn a new syntax, tooling workflow, and infrastructure management paradigm. By using Go instead, we were able to reduce the learning curve, streamline development, and keep the infrastructure management process closely integrated with the rest of the application stack.

1.4 Bootstrapping

Server bootstrapping is a critical part of deployment, as misconfigurations can cause downtime, unstable services, and security vulnerabilities. To reduce these risks, we used Ansible for automated server provisioning and configuration.

Ansible is widely adopted for infrastructure automation and offers an idempotent design, allowing playbooks to run repeatedly without creating inconsistencies. This ensures the server setup remains reproducible, maintainable, and consistently updated while reducing manual errors and improving security.

The setup consists of two Ansible playbooks. The first handles server bootstrapping and security hardening, including system configuration, security measures, and deployment preparation.

Table 1: Security hardening measures applied during server bootstrapping

Security Measure	Description
Non-root administrative user	Creates a user without direct root access while granting <code>sudo</code> privileges.
System upgrades	Updates and upgrades server packages and system components.
Disable root login	Prevents direct root login to improve security.
System hardening	Applies Linux kernel, SSH, and network hardening settings to reduce attack surfaces and improve security.
UFW and Fail2Ban	Configures firewall rules and intrusion prevention mechanisms.
Auditd setup	Enables system auditing and security event logging.
Unattended upgrades	Configures automatic installation of security updates.

The second playbook focuses specifically on installing and configuring Docker along with the Docker Compose plugin. Separating these responsibilities into distinct playbooks improves maintainability and makes the provisioning process easier to understand and manage.

Detailed instructions for executing the playbooks are provided in the repository’s `README` file. This documentation ensures that the setup process is reproducible and accessible for both current and future team members.

Monitoring

Design and architecture

Our metrics pipeline has three components: the Go web service produces metrics (‘GET /metrics’), Prometheus pulls and stores them, and Grafana visualizes them. A fourth service, node-exporter, exposes host-level metrics (CPU, memory, disk). We used the official ‘prometheus/client_golang’ library. Our HTTP middleware automatically records request count, latency, and status code for every route. The counters for specific stats are incremented separately inside the relevant handlers and exposed as, for instance, ‘new_registered_users’. Prometheus then scrapes the metrics every 15s, and Grafana pulls it and visualizes it. The Grafana dashboard is fully provisioned from a folder in our repository, which defines what the dashboard looks like. It is set up automatically after deployment, with the tradeoff that any changes from the UI will not be persistent. Lastly, Caddy exposes Grafana at ‘grafana.kulturbase.dk’ over HTTPS.

What we monitor

The main metrics we monitor are request rate, error rate, latency, and success rate. The reason behind this is that it immediately tells us whether something is wrong, and if so, points us in that direction. The second part of the dashboard focuses on overall health and serving capabilities. The main visualizations show the CPU load, health check, and an overview of activity on the server. This enables us to distinguish app issues from infrastructure issues and whether the app is being actively used.

Reflections

The monitoring setup needs to take into consideration the deployment strategy. When switching from ‘docker compose’ to ‘Docker Swarm’ with multiple replicas, our scraping silently broke. Each scrape hit a different replica through Swarm’s load-balancing virtual IP, so counters appeared to reset between scrapes. The solution was to use the Prometheus DNS service discovery to scrape each replica separately, and then sum the results in Grafana.

Logging

Design and architecture

Our logging pipeline uses an EFK stack (Elasticsearch, Filebeat, Kibana), as presented in the exercises. The Go web service writes structured JSON logs through ‘slog’ to stdout. Docker captures container stdout into ‘/var/lib/docker/containers/’ via its default JSON-file driver. Filebeat runs one instance per node in swarm, tails those log files, decodes the Docker JSON, and ships entries directly to Elasticsearch, which stores and indexes them. Kibana sits in front for search and visualization. The whole stack runs on its own elk overlay network, with Caddy exposing ‘kibana.kulturbase.dk’ over HTTPS.

What we log

The Go app produces structured JSON logs at multiple levels through slog. The HTTP middleware automatically logs every request with method, route, status, duration, and client IP. Handler-level ‘Logger.Error’ calls add explicit context when something fails (DB query, template render, JSON decode). Container stdout from supporting services (Postgres, Caddy, Prometheus) is also captured automatically. In Kibana, we built a dashboard with four saved searches: live error stream, failed-request stream (‘status >= 400’), user-activity stream (registrations, logins, posts), and overall log volume by service.

Reflections

Choosing EFK stack over Grafana + Loki was, in our opinion, the wrong call for a project this small. Elasticsearch alone consumes a large amount of memory, and the stack was tricky to set up. For example, Filebeat refused to start until its config file was owned by root with the right permissions, which broke our deploys until we fixed the file ownership manually on the server. Loki would have let us keep everything under one Grafana frontend, removing Kibana as a separate system to configure and share access to. Even though it introduced unnecessary complexity, the decision was still worth it as a learning experience, as we got to try the industry-standard logging stack and ran into the kinds of real-world problems. For a project of this size it was overkill, but as a one-time exercise to engage with the heavier tools, it was a good choice. It also nicely separated the two concerns - Grafana telling us whether something is wrong, and Kibana explaining what happened.